TypeScript everywhere: Eine neue Ära von Skripten mit Deno

TypeScript everywhere: Eine neue Ära von Skripten mit Deno oder Node

TypeScript everywhere

- Web standards are most open and competitive
  - Lots of libraries, runtime engines
- JavaScript/TypeScript ubiquitous
  - First app of many is written in JavaScript/TypeScript, HTML, CSS
  - Everybody knows JavaScript/TypeScript
  - Runs on every platform
- This talk is especially about what possibilities TypeScript opens up for scripts
  - More to expand horizon and give ideas, not so much a deep dive

TypeScript everywhere

- JavaScript Runtime Environment Overview
- Deno (with examples)
- When to use Deno
- Comparison of languages for scripts
  - Bash, Groovy, JavaScript, TypeScript
- Node.js is catching up
  - Recent developments of Node.js
- Advantages of using Deno for Scripts
- Discussion and further examples

JavaScript Runtime Environment Overview

	Node.js	Deno	Bun	AWS LLRT
Since	2009	2018	2022	2024
Runtime Engine	V8	V8	JavaScriptCore	QuickJS
Written in	JavaScript, C++, Python, C	TypeScript, JavaScript, Rust, C++	Zig, C++, TypeScript	Rust, JavaScript, TypeScript
Runs TypeScript directly	~~no~~ (experimental) --experimental-strip-types default since Node 23	yes	yes	
package manager (npm, yarn, pnpm…) included	no	yes	yes	
separate library file (e.g. package.json) optional	no	yes	no	
secure	~~no~~ `--permission` since Node 20, stable since 23	yes (by default)	no (different builds planned)	
Website	https://nodejs.org/	https://deno.com/	https://bun.sh/	https://github.com/awslabs/llrt

These are for servers, in general there are more:
Chrome (V8), Edge (V8), Firefox (SpiderMonkey), Safari (JavaScriptCore), React Native (Hermes), …

Deno

- Like Node.js: Runtime for JavaScript **and TypeScript**
- Also like Node.js: co-created by Ryan Dahl
- Key Takeaways:
  2. URLs for loading dependencies.
  3. Does not require a package manager like npm.
  4. Runs TypeScript out of the box.
  6. Restricts file system and network access by default to run sandboxed code.

  See https://en.wikipedia.org/wiki/Deno_(software)

  10 Things I Regret About Node.js - Ryan Dahl - JSConf EU
  https://www.youtube.com/watch?v=M3BM9TB-8yA
- https://deno.com/

When to use Deno

- Most *npm:* and *node:* dependencies work out of the box
- Not sure about bigger projects yet
  - Maybe Node.js catches up and supports most features of Deno
- Use Deno for Scripts (instead of bash or JavaScript)
  - Run maintainable TypeScript directly
  - #! deno run and mark as executable

Important properties for scripts

- No build/compile steps
- Self contained
- Execute like an executable file

Comparison of languages for scripts

	Bash	Groovy	JavaScript	TypeScript	
Run	bash script.sh	groovy script.groovy	node script.js	ts-node script.ts	deno run script.ts
Library managment			npm, yarn, pnpm, ...		not needed
Library versions			package.json, node_modules		Part of import or deno.lock
Type safety	no	no	no	yes	yes
Sandboxing / secure by default			no	no	yes

* Node.js caught up and since version 23 --experimental-strip-types is default and Node.js can run TypeScript directly as well

Node.js is catching up

- TypeScript support
  - Node.js added `--experimental-strip-types`
  - Experimental since v22.6.0 (August 2024)
  - Stable and default since v23.6.0 (January 2025)
    - `--no-experimental-strip-types` to prevent type stripping
  - Replaces TypeScript syntax with whitespaces
  - (`--experimental-transform-types` for TypeScript-only syntax like enums)
  - https://nodejs.org/api/typescript.html#type-stripping

- Node.js added `--experimental-network-imports`
  - But removed it in v22.6.0 (August 2024)
  - https://github.com/nodejs/node/pull/53822

- Security
  - Node.js has now a Permission Model
  - Experimental since v20.0.0 (April 2023)
  - Stable since v23.5.0 (December 2024)
  - Example: `node --permission --allow-fs-read=/folder/ script.ts`
  - https://nodejs.org/api/cli.html#--permission

Advantages of using Deno for Scripts

- Better maintainable than Bash, Groovy or JavaScript but as simple
  - No config, no package.json, no build step required
- TypeScript is commonly used by developers
- Deno can be used via npm:
  - npm install --save-dev deno-bin
  - https://www.npmjs.com/package/deno-bin
- Examples used in Demo:
  - https://github.com/MichiSpebach/denoexamples